

SCOUT: Structure-Aware Aspect and Anchor-Count Selection for Node Attribute Augmentation via Positional Information

Dong-Hyuk Seo
Department of Computer Science
Hanyang University
Seoul, Republic of Korea
hyuk125@hanyang.ac.kr

Sein Kim
Department of Computer Science
Hanyang University
Seoul, Republic of Korea
seinkim@hanyang.ac.kr

Taeri Kim
Department of Computer Science
Hanyang University
Seoul, Republic of Korea
taerik@hanyang.ac.kr

Won-Yong Shin
Computational Science and
Engineering, Yonsei University
Seoul, Republic of Korea
wy.shin@yonsei.ac.kr

Sang-Wook Kim*
Department of Computer Science
Hanyang University
Seoul, Republic of Korea
wook@hanyang.ac.kr

Abstract

In the absence of node attributes, Graph Neural Networks (GNNs) often fail to distinguish locally isomorphic nodes, leading to suboptimal performance. To compensate for this, Positional Information (PI) augmentation has emerged as a powerful technique, which generates attributes by selecting representative nodes as anchors and encoding node-to-anchor distances to other nodes. However, the performance of PI-based methods hinges on two graph-dependent choices: 1) the structural measures used for anchor selection and distance metrics, and 2) the anchor-count K . To obviate manual selections, we propose SCOUT, a model-agnostic augmentation framework that learns a graph-level selector to identify the optimal structural measure and adaptively determines the anchor-count K tailored to each graph and task. Subsequently, leveraging the heavy-tailed distribution typically observed in node centrality, SCOUT utilizes an elbow detection method on the ranked centrality curve to adaptively determine the K most representative nodes as anchors. SCOUT is model-agnostic and enhances various GNNs across downstream tasks. It achieves an improvement of 26.88% in Hits@20 for link prediction on ogbl-ddi and 4.52% accuracy points for node classification on ogbn-arxiv without original attributes; with original attributes, it also brings additional gains of 6.15% AUC on Cora and 11.69% accuracy points on ogbn-arxiv. The source code of SCOUT is available at <https://github.com/seinkim01/SCOUT.git>.

CCS Concepts

• Theory of computation → Graph algorithms analysis.

Keywords

Anchor, graph neural network, node attribute augmentation, positional information, structure awareness

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792326>

ACM Reference Format:

Dong-Hyuk Seo, Sein Kim, Taeri Kim, Won-Yong Shin, and Sang-Wook Kim. 2026. SCOUT: Structure-Aware Aspect and Anchor-Count Selection for Node Attribute Augmentation via Positional Information. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792326>

1 Introduction

Graph-structured data are ubiquitous in real-world applications, including recommender systems [23, 24, 39, 55], drug discovery [13, 57, 65], and traffic prediction [40, 52, 60]. Graph Neural Networks (GNNs) [49, 53, 62] have emerged as the de facto architecture for analyzing graphs. Via the message-passing mechanism [14, 17, 21, 48], GNNs aggregate multi-hop neighbors [44] to learn node representations that jointly encode node attributes (*i.e.*, features) and graph topology.

However, in many real graphs, node attributes are often unavailable due to privacy constraints or collection costs (*e.g.*, omitted user profiles or patient records [7, 37, 45, 66]). In the absence of attributes, GNNs rely solely on topology and struggle to distinguish locally isomorphic nodes¹ [32], which substantially degrades performance of downstream tasks. Prior work has therefore explored attribute augmentation using structural information, including random or constant initializations [9], single structural measures (*i.e.*, degree centrality [41], PageRank [6], betweenness centrality [11]), and structure-aware labeling schemes such as IDGNN [56] and SEAL [43, 61].

Among attribute augmentation methods, positional information (PI) approaches [22, 30, 58, 64] have become prominent. Fig. 1 illustrates the standard pipeline of PI-based methods. They represent a node's relative position using distances to other nodes; however, encoding distances to all nodes incurs quadratic costs. [26, 59] To avoid this, PI-based methods select a small subset of nodes, called anchors, that summarize the underlying graph and then compute each node's distances to those anchors. The resulting node-anchor distances constitute the PI and serve as the augmented attributes for GNNs, thereby sharpening their discriminative power.

¹Two nodes are locally isomorphic if they lie in different regions of the graph yet share identical local topology.

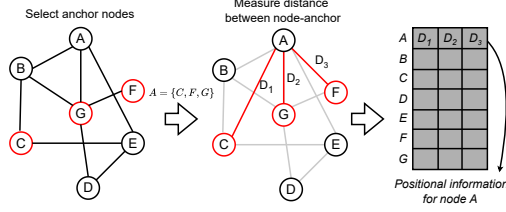


Figure 1: Pipeline of standard PI-based methods: select anchors using a centrality measure and compute an anchor-node distance using a node-pair similarity measure as augmented attributes.

Despite their effectiveness, PI-based methods hinge on two dependencies: (D1) dependence on structural measures and (D2) sensitivity to the anchor-count K (i.e., the number of anchors).

(D1) Dependence on structural measures. A PI pipeline consists of (i) an anchor-selection criterion and (ii) a node-anchor distance, which are typically instantiated by structural measures: a centrality measure and a node-pair similarity measure, respectively. High-degree anchors [22] or sample anchors uniformly at random [30, 58] are commonly chosen, while distances are often set to the score of shortest paths [4], random-walk-based similarity [15, 34], or Jaccard similarity [20]. Because different centralities and similarities reflect different structural signals [36], each centrality-similarity pair induces a distinct structural aspect; for convenience, we refer to this induced aspect as a **positional aspect**.

This dependence is evident in Fig. 2(a), which fixes the PI-based method (P-GNN) and shows that accuracy changes substantially across anchor selectors, with different winners across datasets. We observe the same pattern for the similarity measure (see Appendix E). Therefore, PI should avoid committing to a single measure; instead, it should *pool multiple positional aspects and adaptively pick the one that best explains a given graph*.

(D2) Dependence to the anchor-count. Performance strongly depends on the anchor-count K . Too few anchors miss positionally informative nodes, whereas too many anchors increase computations, creating an inherent trade-off. Many methods set $O(\log N)$ anchors [58], motivated by Bourgain’s theorem [5]; however, this serves as a heuristic guideline rather than an optimal choice. Fig. 2(b) evaluates the performance of P-GNN [58] by varying the anchor-count K over a range around the commonly used setting (e.g., $K = 64$), revealing substantial performance variation across K . Within a dataset, the accuracy curve is non-monotonic and often peaks away from $K = 64$; across datasets, the optimal K also shifts. These results indicate that anchor-count directly affects task performance and thus calls for a *graph-aware K selection strategy*.

To address (D1) and (D2), we propose *Selection of positional aspects and anchor count for attribute augmentation* (SCOUT), an adaptive and selective positional-attribute augmentation method. Our key challenges are (C1) to represent diverse positional aspects while selecting the most suitable structural measures per graph, and (C2) to choose the anchor-count in a graph-specific manner. SCOUT follows the standard PI recipe, which first selects anchors via a centrality measure and defines the anchor-node distance via a node-pair similarity measure described above.

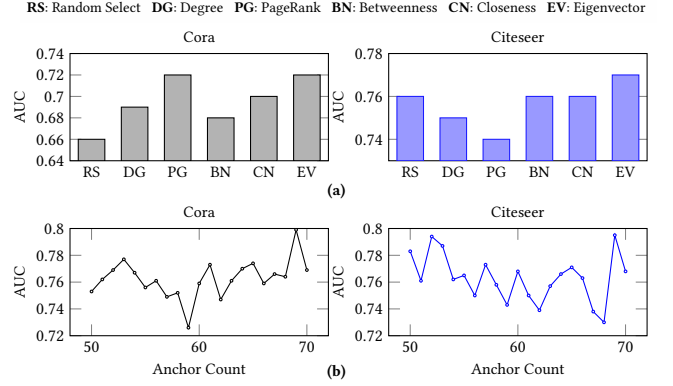


Figure 2: (a) Fixing the PI method to P-GNN, performance varies across anchor selectors, and the winner changes across datasets. (b) Performance is non-monotone in K , and the optimal K shifts across datasets. SCOUT selects both the positional aspect and K in a graph-aware manner.

For (C1), we first construct attributes that encode multiple positional aspects by instantiating PI with diverse centrality-similarity pairs. We then learn a graph representation and use it to graph-level aspect selection module, selecting the graph-level aspect that best matches the current graph and task, and apply it consistently to all nodes in the graph. For (C2), we leverage the classical empirical regularity that centrality scores are often heavy-tailed and approximately follow a power-law distribution [8]. Such distributions assign very high scores to a small set of nodes while most nodes score low. We posit that these high-centrality nodes are positionally informative and thus suitable anchors [42]. To this end, we set K to the elbow point of the ranked centrality curve via a standard elbow method [38].

We validate the superiority of SCOUT through extensive experiments. On OGB benchmarks without original attributes, SCOUT improves Hits@20 for link prediction on ogbl-ddi by **26.88%** and accuracy for node classification on ogbn-arxiv by **4.52%**, demonstrating that SCOUT provides effective node-attribute augmentation for standard graph downstream tasks. With original attributes, SCOUT improves AUC for link prediction on Cora by **6.15%** and accuracy for node classification on ogbn-arxiv by **11.69%**, confirming its utility in attribute-present regimes as well. All improvements are reported as relative gains over the best-performing baseline under the same setting. Finally, additional analyses verify that (i) our structural-measure selection mechanism identifies informative measures and (ii) our elbow-based criterion chooses appropriate anchor-counts for each graph.

Our contributions are summarized as follows:

- We identify two graph-dependent failure modes of PI augmentation—*measure dependency* and *anchor-count sensitivity*—and validate both with controlled studies.
- We propose SCOUT, which **selects the positional aspect** via graph-level selection and **chooses the anchor-count K** via an elbow-based rule grounded in power-law centrality.
- We demonstrate consistent gains across diverse graphs and tasks, in both cases with and without node attributes.

2 Related Work

GNNs learn node representations by aggregating information over local neighborhoods [13]. When node attributes are missing, GNNs struggle to distinguish locally isomorphic nodes, which degrades downstream accuracy [31]. This limitation motivates augmenting structural as node attributes [61].

Structural based. Early works replace missing attributes with handcrafted structural measures such as degree centrality [41], PageRank [6], and betweenness centrality [11]. These signals are lightweight, yet each different measure reflects a different aspect of structure and remains sensitive to heuristic choices [19, 58]. Our approach moves beyond single structural scores by encoding node–anchor similarities that capture long-range positional context.

Labeling-based. Labeling schemes like DRNL and subgraph extraction methods such as SEAL provide task-aware structural labels and strong link-prediction baselines [1, 51]. These methods excel on localized structures, yet they incur higher subgraph costs and can miss global positional cues [47, 61]. Our method leverages PI by encoding relations to anchor nodes, thereby supplying global signals beyond local neighborhoods.

PI-based. Anchor-based PI encodes each node by its similarity to a selected set of anchors, trading fidelity for efficiency [58]. However, accuracy depends on two graph-dependent choices, the structural measures for anchor selection and node–anchor distance, and the anchor-count K [19]. We unify multiple centrality–similarity pairs into a single multi-aspect attribute and select the informative aspects at the graph-level.

3 Proposed Method

3.1 Overview

SCOUT addresses the performance limits of GNNs on graphs without node attributes by learning the attribute augmentation method. Grounded in positional attribute augmentation, SCOUT addresses (D1) its dependence on structural measures and (D2) its sensitivity to the anchor-count by learning graph-specific selections, achieving dataset-robust performance.

We therefore pose three challenges that refine (D1) and (D2):

- **C1-1.** Different centrality–similarity pairs reveal different positional aspects. How can we augment attributes that capture multiple positional aspects at once? (D1)
- **C1-2.** The most informative positional aspect is graph-dependent, so how can we learn to select the right aspect for each graph? (D1)
- **C2.** The optimal anchor-count K varies across graph, so how can we set K adaptively? (D2)

We resolve these challenges with three components:

- **C1-1.** We design augmented attributes that aggregate complementary PI aspects (Section 3.2).
- **C1-2.** We learn graph-aware aspect selection with an attention module that has weights shared across nodes. (Section 3.3).
- **C2.** We set K via an elbow rule grounded in the power-law distribution of centrality (Section 3.4).

Fig. 3 illustrates the architecture of SCOUT. For a simple exposition, we fix three centrality measures and three similarity measures.

Given a graph, we first rank nodes by each centrality and select K anchors, where K is determined by our elbow method. For each node–anchor pair, we compute similarities, producing a set of vectors $\{S^{(i,j)}\}$ where i indexes centrality and j indexes similarity. We treat each $\{S^{(i,j)}\}$ as a positional aspect and stack the corresponding $S_{i,j}$ to form the multi-aspect positional attribute $\hat{X}$. A graph-level aspect-selection module then assigns shared attention weights to aspects, using a global query and per-aspect keys to emphasize informative aspects while suppressing noisy ones. We apply aspect-wise aggregation to obtain the selected attribute $\tilde{X}$. When original node attributes exist, we concatenate $\tilde{X}$ with the originals; otherwise we feed $\tilde{X}$ alone into a backbone GNN. The backbone is trained end-to-end for link prediction and node classification, so the selection module learns to surface aspects that most improve downstream accuracy.

We summarize the notation used in this section and throughout the paper in Appendix A. For vector or matrix indexing, we use the Matlab-like notation: X_i denotes the (i) -th element of vector X .

3.2 Attributes for Multi-Aspect PI

PI-based methods select anchors by a centrality measure and encode node–anchor distance by similarities measure as PI. Prior work fixes these measures heuristically. We observe that task accuracy varies widely with this choice (see Section 1). This is because different centrality–similarity pairs capture different positional aspects, and the most useful aspect depends on the graph and the task. Hence we need a mechanism that both (i) encodes multiple aspects and (ii) selects which aspect to leverage for the current graph and task. This subsection specifies a node-attribute form that encodes multi-aspect structural information; the aspect selection mechanism follows in Section 3.3. Throughout Section 3.2 and Section 3.3 we assume a per-graph anchor-count K shared across all centralities; Section 3.4 details how K is chosen.

We now make this multi-aspect PI concrete by aggregating node–anchor similarities across centralities and similarity measures. We consider ℓ centralities $c^{(i)} : V \rightarrow \mathbb{R}_{\geq 0}$ for $i \in [\ell]$, where larger values indicate higher centrality. We also consider m node-pair similarities $s^{(j)} : V \times V \rightarrow \mathbb{R}_{\geq 0}$ for $j \in [m]$, where larger values indicate greater similarity. Using the anchor-count rule in Section 3.4, we select K anchors for each centrality. For each $i \in [\ell]$, define anchor set

$$R^{(i)} = \{r_1^{(i)}, \dots, r_K^{(i)}\} \subseteq V, \quad (1)$$

which contains the K anchors associated with $c^{(i)}$. We use $p \in [K]$ to index anchors within $R^{(i)}$.

We instantiate PI by aggregating node–anchor similarities across all aspects. For a node $n \in V$ and an anchor $r_p^{(i)} \in R^{(i)}$ where $p \in [K]$, the basic positional signal for centrality measure i and similarity measure j is

$$s^{(j)}(n, r_p^{(i)}). \quad (2)$$

Collecting the similarities from n to all K anchors in $R^{(i)}$ yields

$$S^{(i,j)}(n) = [s^{(j)}(n, r_1^{(i)}), s^{(j)}(n, r_2^{(i)}), \dots, s^{(j)}(n, r_K^{(i)})] \in \mathbb{R}^K. \quad (3)$$

The vector $S^{(i,j)}(n)$ is one aspect of PI of node n under the {centrality i , similarity j }.

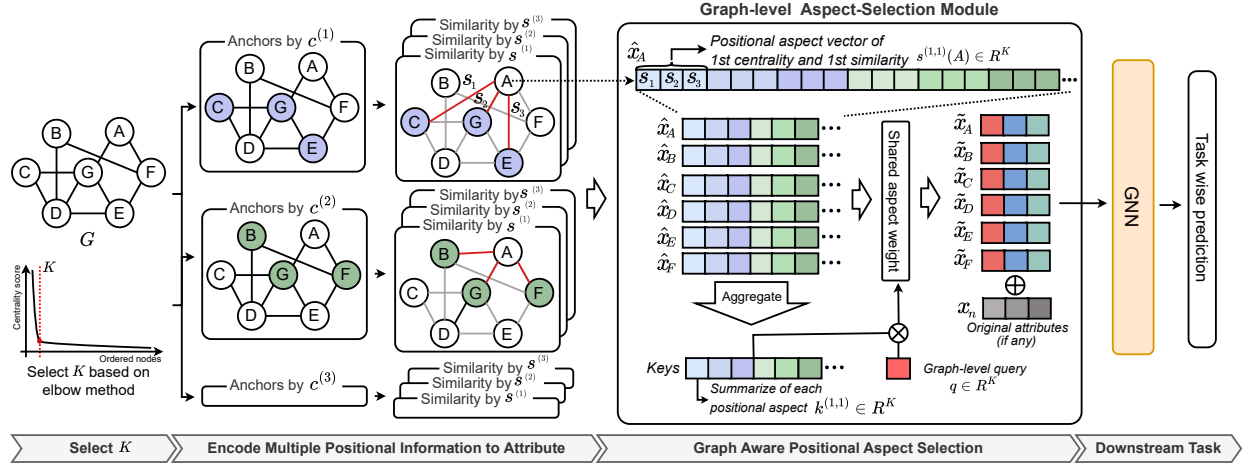


Figure 3: The overview of SCOUT. We first set the anchor-count K , detected on the ranked centrality curve using the elbow rule, adapting K to each graph. Second, encode multi-aspect PI by stacking node-anchor similarities over centrality-similarity pairs, yielding $\hat{x}_n \in \mathbb{R}^{mK}$. A graph-level aspect-selection module then applies shared attention weights, computed from a graph-level query vector q and per-aspect keys $k^{(i,j)}$, to obtain the selected attribute $\tilde{x}_n$. Finally, $\tilde{x}_n$ is concatenated with original attributes (if any) and fed to a backbone GNN for link prediction or node classification.

We then stack over the m similarities to form the per-centrality vector

$$P^i(n) = [S^{(i,1)}(n) \parallel S^{(i,2)}(n) \parallel \dots \parallel S^{(i,m)}(n)] \in \mathbb{R}^{mK}. \quad (4)$$

Finally, stacking over the ℓ centralities to obtain the node attribute

$$\hat{x}_n = [P_1(n) \parallel P_2(n) \parallel \dots \parallel P_\ell(n)] \in \mathbb{R}^{\ell m K}. \quad (5)$$

By construction, $\hat{x}_n$ preserves PI from every aspect of centrality-similarity pair.

3.3 Graph-Level Aspect-Selection Module

We now determine which aspects of PI to leverage. The vector $\hat{x}_n$ defined in Section 3.2 encodes all positional aspects, and such overcomplete information can hurt the task. We therefore introduce a module that learns to retain only the informative aspects. To enable data-driven selection, we use an attention mechanism [50] that produces a weighted aggregation over positional aspects.

In this setting, link prediction and related tasks require fair comparison across nodes; the same positional aspect must be selected for all nodes in a graph. We thus enforce **graph-level selection** by sharing one distribution over aspects across nodes, which preserves consistency and reduces node-wise overfitting. Concretely, we learn a graph vector that represents the input graph and aspect-level vectors that summarize each positional aspect. We score each aspect by the compatibility between the graph vector and the aspect vector, and obtain weights shared by all nodes. We aggregate positional attributes with these shared weights and feed the result to the backbone GNN.

Concretely, an *aspect* of PI corresponds to a centrality-similarity pair (i, j) , i.e., the vector $S^{(i,j)}(n) \in \mathbb{R}^K$ defined in equation 3 at Section 3.2. We use two linear projections $\mathbf{W}_{val}, \mathbf{W}_{key} \in \mathbb{R}^{K \times K}$ for

attention mechanism. For each node n , we form the value representation

$$\mathbf{v}^{(i,j)}(n) = \mathbf{W}_{val} S^{(i,j)}(n) \in \mathbb{R}^K. \quad (6)$$

To summarize each aspect at the graph-level, we average its blocks over nodes

$$\bar{S}^{(i,j)} = \frac{1}{|V|} \sum_{n \in V} S^{(i,j)}(n) \in \mathbb{R}^K, \quad \mathbf{k}^{(i,j)} = \mathbf{W}_{key} \bar{S}^{(i,j)} \in \mathbb{R}^K, \quad (7)$$

which yields a key $\mathbf{k}^{(i,j)}$ that represents the (i, j) aspect for the current graph.

We parameterize a **single graph-level query** $\mathbf{q} \in \mathbb{R}^K$ that is randomly initialized and learned end-to-end with the task objective. Dot-product attention produces logits and shared aspect weights

$$a^{(i,j)} = \frac{\mathbf{q}^\top \mathbf{k}^{(i,j)}}{\sqrt{K}}, \quad w^{(i,j)} = \frac{\exp(a^{(i,j)})}{\sum_{i'=1}^{\ell} \sum_{j'=1}^m \exp(a^{(i',j')})}. \quad (8)$$

For each node n , we aggregate values with the shared weights to obtain the selected positional attribute

$$\tilde{x}_n = \sum_{i=1}^{\ell} \sum_{j=1}^m w^{(i,j)} \mathbf{v}^{(i,j)}(n) \in \mathbb{R}^K. \quad (9)$$

Because $w^{(i,j)}$ depends only on $\mathbf{q}$ and $\mathbf{k}^{(i,j)}$, the same aspect selection applies to every node, which ensures a comparable selection at the graph-level.

In summary, the attention mechanism aggregates aspect blocks into a graph-specific node embedding by scoring keys $\mathbf{k}^{(i,j)}$ against a global query $\mathbf{q}$ and weighting the corresponding values $\mathbf{v}^{(i,j)}(n)$. This yields graph-level weights $\{w^{(i,j)}\}$ that emphasize the structural aspects most useful for the task while applying the same

mixture to all nodes. The attention parameters are optimized end-to-end through the task loss; the predictor and training objective are presented in Section 3.5.

3.4 Selecting an Appropriate Anchor-Count K

The anchor-count K controls the resolution of PI. Using all nodes as anchors yields maximal fidelity but costs $O(|V|^2)$, so PI-based methods sample K anchor instead. The anchor-count K is often set near $O(\log|V|)$ based on Bourgain’s theorem [5]. However, this is a heuristic rather than an optimal choice, and—as observed in our study—task accuracy varies markedly with K and the optimal K shifts across graphs.

We propose a graph-aware rule that sets K from the empirical distribution of centrality scores. In many real graphs, centrality scores are heavy-tailed and approximately power-law [8], with a few nodes scoring very high and most scoring low. This pattern suggests that only a small, graph-dependent fraction of high-centrality nodes is structurally salient. In this work, we focus on the elbow method as a simple, data-driven rule. The elbow method finds the “elbow point” of a ranked curve, i.e., the point of the sharpest slope change. [38] We apply this elbow rule to the ranked centrality curve and set K to the elbow point. This adapts K per graph and concentrates anchors in the high-centrality regime.

Concretely, we adapt the Kneedle algorithm—one of the standard methods for elbow (knee) detection [38]. We first construct a ranked centrality curve. Let $V = \{v_1, \dots, v_N\}$ and $c : V \rightarrow \mathbb{R}_{\geq 0}$ be a centrality measure. From V and c we form the score vector $\mathbf{c} = (c(v_1), \dots, c(v_N))$. Let π be a permutation of $\{1, \dots, N\}$ such that $c(v_{\pi(1)}) \geq \dots \geq c(v_{\pi(N)})$. Define the ranked scores (and nodes) by

$$s_i := c(v_{\pi(i)}), \quad i = 1, \dots, N, \quad (10)$$

and write $S = \{s_1, \dots, s_N\}$ with $s_1 > 0$. We normalize ranks and scores to $[0, 1]$ and form an increasing curve for Kneedle:

$$\begin{aligned} x_i &= \frac{i-1}{N-1}, & y_i &= \frac{s_i}{S_1} \in [0, 1], \\ g_i &= 1 - y_i \in [0, 1], & i &= 1, \dots, N. \end{aligned} \quad (11)$$

Here x_i is the normalized rank, y_i is the normalized score, and g_i is the increasing curve used by Kneedle. Kneedle operates on the difference between this curve and the identity:

$$D_i := g_i - x_i, \quad i = 1, \dots, N. \quad (12)$$

D_i measures how far the ranked curve lies above the identity at rank i . We take local maxima of D as knee candidates and confirm a knee after a candidate once D drops by a fixed margin:

$$\mathcal{M} := \{i \in \{2, \dots, N-1\} \mid D_{i-1} < D_i > D_{i+1}\}, \quad (13)$$

$$T_m := D_m - \tau, \quad \tau := \lambda \cdot \left(\max_j D_j - \min_j D_j \right), \quad \lambda \in (0, 1), \quad (14)$$

$$k(m) := \min\{j > m \mid D_j \leq T_m\}, \quad k := \min\{k(m) \mid m \in \mathcal{M}\}. \quad (15)$$

Here τ is a sensitivity level set as a fraction $\lambda \in (0, 1)$ of the range of D . We set the anchor-count by

$$K := k, \quad (16)$$

and if no knee is detected for the chosen τ , we use $K = \arg \max_i D_i$.

We employ multiple centrality measures to capture complementary PI aspects. When multiple centralities $\{c^{(i)}\}_{i=1}^\ell$ are used, repeating the above yields knees $\{k_i\}$. For simplicity and stability, we aggregate them by the rounded mean:

$$K := \text{round}\left(\frac{1}{\ell} \sum_{i=1}^\ell k_i\right). \quad (17)$$

We show that using per-measure k_i versus the aggregated K attains similar accuracy in Appendix F.

3.5 Training and Prediction

Our augmented node attribute is used as the initial attribute of a GNN backbone. The method is model agnostic so any graph neural network model can be used, such as GCN [62] or GraphSAGE [17].

Let $X^{\text{orig}} \in \mathbb{R}^{N \times d}$ denote the original node attribute matrix if available. Let $\tilde{\mathbf{x}}_n \in \mathbb{R}^h$ be the attention-aggregated positional attribute from Section 3.3. We form the GNN input as

$$\mathbf{h}_n^{(0)} = \begin{cases} [\mathbf{x}_n^{\text{orig}} \parallel \tilde{\mathbf{x}}_n], & \text{if original attributes exist,} \\ \tilde{\mathbf{x}}_n, & \text{otherwise,} \end{cases} \quad (18)$$

where $\mathbf{x}_n^{\text{orig}}$ is the n -th row of X and $\parallel$ denotes concatenation.

A L -layer GNN transforms $\mathbf{h}_n^{(0)}$ into a node embedding $\mathbf{z}_n \in \mathbb{R}^{d_z}$. We write this compactly as

$$\mathbf{Z} = \text{GNN}_\theta(\mathcal{G}, \{\mathbf{h}_n^{(0)}\}_{n \in V}), \quad \mathbf{z}_n \text{ is the } n\text{-th row of } \mathbf{Z}, \quad (19)$$

and we train the GNN jointly with the attention module in an end-to-end manner.

Link prediction. For a pair (u, v) , we score the link with a small MLP on the concatenated embeddings

$$s_{uv} = \text{MLP}_\psi([\mathbf{z}_u \parallel \mathbf{z}_v]), \quad \hat{y}_{uv} = \sigma(s_{uv}), \quad (20)$$

and we optimize binary cross entropy on positive edges $\mathcal{E}^+$ and sampled negatives $\mathcal{E}^-$.

Node classification. For C classes we compute logits by

$$\mathbf{o}_n = \text{MLP}_\phi(\mathbf{z}_n) \in \mathbb{R}^C. \quad (21)$$

For single-label classification, we minimize the cross-entropy over the labeled set $\mathcal{V}_L$. For multi-label classification, we replace the softmax with elementwise sigmoid and use binary cross-entropy.

4 Experiments

4.1 Research Questions

We evaluate SCOUT through extensive experiments and organize the study around the following research questions (RQs):

- **RQ1.** Does SCOUT improve accuracy over node-attribute augmentation baselines on link prediction and node classification, both without and with original attributes?
- **RQ2.** Are diverse positional aspects effective over a single aspect, and does *graph-level aspect selection* module further improve over naive multi-aspect attributes?
- **RQ3.** Does *graph-level aspect selection* of SCOUT outperform node-level selection and element selection in accuracy?
- **RQ4.** Does the elbow rule-based select an anchor-count K that matches the empirical optimum?

- **RQ5.** How does the computational cost of SCOUT compare with competing augmentation methods?

4.2 Experimental Setup

Datasets, Tasks, and Metrics. We evaluate on link prediction and node classification, both without and with original attributes. We consider three *small* citation graphs and two *large* OGB graphs. The small graphs are *Cora* [29], *Citeseer* [12], and *Pubmed* [33]. The large graphs are *ogbl-ddi* for link prediction and *ogbn-arxiv* for node classification [18, 25]. On citation graphs, we split edges for link prediction and nodes for node classification into train/validation/test with ratios 0.7/0.1/0.2. On OGB graphs, we adopt the official splits to ensure comparability [18]. For link prediction on citation graphs we report AUC as used by prior work [61]. For *ogbl-ddi*, we report Hits@20 because the benchmark defines it as the official metric and most studies follow it [18, 22]. For node classification we report accuracy on all datasets [25]. We construct augmented attributes using training nodes and edges only and apply the same protocol to all baselines.

Baselines. We compare against no-feature, learning-based, labeling, structural, and positional-information methods under a unified protocol. We keep the prediction head, optimizer, and schedule identical across methods, and we construct any structural or positional attributes using training edges only for fair comparison.

- **Ad-hoc.** *Constant* uses all-one features to test topology-only capacity. *Random (fixed)* samples random features once and freezes them. *Random (learnable)* initializes features randomly and learns them end-to-end with the backbone.
- **Learning-based.** *node2vec* [15] provides unsupervised random walk-based embeddings that we use as node attributes, trained on training edges only.
- **Labeling.** *DRNL* [61] uses double-radius node labeling as structural attributes. *SEAL* [61] extracts local subgraphs around candidate links and applies DRNL labels with a GNN decoder, serving as a strong link-prediction baseline.
- **Structural measures.** *NCSM* [27] constructs a structure measure-based attribute by injecting node centralities and pairwise similarity scores, without using anchors. The simpler variants that use centrality scores as standalone node attributes (e.g., degree, PageRank) are subsumed under it. [6, 11, 41]
- **Positional information.** *P-GNN* [58] encodes node–anchor similarities as PI that integrates with standard GNN backbones. *HPLC* [22] provides high-order positional encoding with learned anchors and similarities as a modern PI-based augmentation.

Backbone GNNs and protocol. To verify SCOUT as an augmentation strategy, we plug it into three GNNs, GCN [62], GRAPH-SAGE [17], and Link-MoE (MoE) [28]. MoE, a mixture-of-experts model that is reported as state-of-the-art on graph benchmark [18]. We concatenate the representation of SCOUT attribute and the representation of original attributes when available; otherwise, the backbone consumes only SCOUT attribute. For fairness, we apply the same protocol to all competing attribute-augmentation methods. We keep the task loss and training schedule identical across

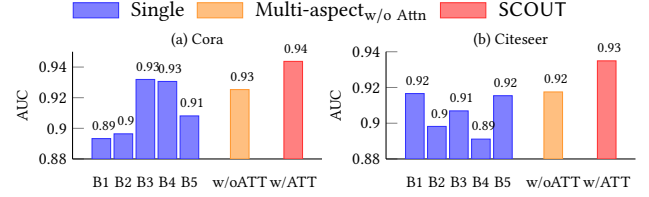


Figure 4: Effects of multi-aspect PI and the aspect-selection module on two datasets. SCOUT combines both components and achieves the highest performance, confirming that the two mechanisms are complementary.

methods. See Section 3.5 for details. Also, additional experiments with newer backbones (e.g., graph transformer) are reported in Appendix G.

Implementation details for reproducibility. SCOUT leverages multi-aspect PI. To instantiate this, we combine four centrality measures—PageRank [6], betweenness [11], closeness [2], eigenvector [3]—and four similarity measures—RWR [34], AdaSim [16], resource allocation (RA) [63], Jaccard [20]—giving $\ell = 4$ and $m = 4$ aspects, respectively. This yields $\ell m = 16$ centrality–similarity aspects per graph. The detailed description of each structural measure is provided in the Appendix.

Experiments run on NVIDIA A6000 GPUs (48 GB). We report the average over 3 seeds unless noted.

4.3 Results

RQ1: Comparison with node attribute augmentation baselines. To validate SCOUT as a node-attribute augmentation, we plug it into three backbones—GCN [62], GraphSAGE [17], and a recent state-of-the-art model (Link-MoE) [28]—and compare against strong baselines. The detailed settings for comparisons are described in Section 4.2.

Table 1 reports AUC on citation graphs for link prediction, Hits@20 on *ogbl-ddi*, and accuracy for node classification. For *ogbl-ddi*, which has no original attributes, we omit the results for the setting with original attributes. Notably, SCOUT consistently outperforms all competitors across datasets and tasks in both settings without and with original attributes. This holds for all three backbones, which shows that our augmentation is model-agnostic. Second, the gains are larger when original attributes are absent and on the large *ogbl-ddi* graph, indicating that SCOUT compensates for weak or missing node features and scale to large graphs. Because privacy constraints reduce accessible node attributes in many graphs [67], SCOUT that remains effective without original attributes becomes increasingly important. Third, within positional-information methods that share the anchor-based paradigm, HLPC [22] is the strongest baseline, yet SCOUT achieves higher accuracy on every backbone. This supports our design that encodes multiple aspects and selects a graph-level, shared aspect rather than relying on a single aspect. Fourth, despite the state-of-the-art performance of LINKMOE, adding SCOUT improves it further, which confirms complementarity to strong backbones.

RQ2: Ablation on Multi-Aspect Attributes and Graph-Level Selection. PI encodes node positions by applying a node–pair similarity between each node and anchors selected by a centrality.

Table 1: Performance comparison on link prediction and node classification tasks with and without original node attributes. SCOUT significantly outperforms all competitors in all cases. That is, SCOUT provides most informative augmented attributes.

Link Prediction (w/o orig. attr)		Cora (AUC)			Citeseer (AUC)			Pubmed (AUC)			ogbl-ddi (Hits@20)		
Type	Augmentation Method	GCN	SAGE	MoE	GCN	SAGE	MoE	GCN	SAGE	MoE	GCN	SAGE	MoE
Ad-hoc	Constant features (all-ones)	0.5880	0.4765	0.5000	0.6125	0.4553	0.5000	0.6711	0.4100	0.5000	0.3993	0.2194	0.0018
	Random init (fixed)	0.8187	0.7858	0.6686	0.7538	0.7896	0.6411	0.7047	0.6903	0.7693	0.5671	0.5123	0.0114
	Random init (learnable)	0.8150	0.7823	0.6972	0.8150	0.7852	0.6351	0.7049	0.6894	0.7812	0.5842	0.5604	0.0133
Learning-based	Node2Vec [15]	0.8393	0.8654	0.7263	0.7941	0.8083	0.6878	0.8365	0.8442	0.7648	0.5906	0.6188	0.0269
Labeling	SEAL [61]	0.8913	0.9248	0.5792	0.8999	0.9164	0.5662	0.8891	0.9140	0.5460	0.2625	0.2321	0.2564
	DRNL [61]	0.6575	0.6100	0.8383	0.7199	0.7105	0.8063	0.5848	0.5019	0.8184	0.4625	0.4103	0.4405
Structural	NCSM [27]	0.6296	0.7347	0.6927	0.6950	0.7256	0.6558	0.6620	0.6849	0.7369	0.0822	0.3430	0.3947
Positional	P-GNN [58]	0.4316	0.4907	0.8216	0.4688	0.4931	0.8438	0.7132	0.4830	0.7924	0.1140	0.0020	0.0190
	HPLC [22]	0.9135	0.9135	0.9443	0.9293	0.9205	0.9080	0.9427	0.9191	0.7377	0.6803	0.6827	0.5750
SCOUT (Ours)		0.9434	0.9498	0.9545	0.9349	0.9276	0.9109	0.9588	0.9421	0.9607	0.7884	0.8662	0.6900
GAIN		+3.27%	+2.71%	+1.08%	+0.60%	+0.77%	+0.32%	+1.71%	+2.50%	+17.39%	+15.89%	+26.88%	+20.00%
Link Prediction (w/ orig. attr)		Cora (AUC)			Citeseer (AUC)			Pubmed (AUC)			ogbl-ddi (Hits@20)		
only original attributes		0.8530	0.8522	0.6393	0.7940	0.8412	0.6355	0.8594	0.8069	0.7192	-	-	-
Learning-based	Node2Vec [15]	0.8962	0.8759	0.6458	0.8999	0.8541	0.6844	0.8450	0.8469	0.8567	-	-	-
Labeling	SEAL [61]	0.9096	0.8992	0.8366	0.8979	0.9218	0.8268	0.8967	0.9186	0.8259	-	-	-
	DRNL [61]	0.8756	0.8889	0.7944	0.8729	0.8855	0.8014	0.8572	0.8862	0.8543	-	-	-
Structural	NCSM [27]	0.8896	0.8656	0.7538	0.8632	0.8267	0.7839	0.9589	0.8710	0.8348	-	-	-
Positional	P-GNN [58]	0.8770	0.8181	0.9038	0.8585	0.8129	0.8841	0.9057	0.8506	0.8777	-	-	-
	HPLC [22]	0.9248	0.9141	0.9091	0.9359	0.9302	0.8880	0.9525	0.9188	0.9598	-	-	-
SCOUT (Ours)		0.9512	0.9316	0.9650	0.9362	0.9346	0.9414	0.9618	0.9451	0.9625	-	-	-
GAIN		+2.85%	+1.91%	+6.15%	+0.03%	+0.47%	+6.01%	+0.30%	+2.86%	+0.28%	-	-	-
Node Classification (w/o orig. attr)		Cora (Acc)			Citeseer (Acc)			Pubmed (Acc)			ogbn-arxiv (Acc)		
Ad-hoc	Constant features (all-ones)	0.3110	0.3190	0.1510	0.2190	0.1830	0.2040	0.4060	0.4070	0.3820	0.2156	0.0586	0.2156
	Random init (fixed)	0.3790	0.2890	0.4100	0.2830	0.2290	0.2800	0.3370	0.3590	0.3620	0.1859	0.1148	0.1884
	Random init (learnable)	0.4790	0.3830	0.5030	0.3340	0.2420	0.3200	0.3720	0.3600	0.3380	0.1879	0.1172	0.1799
Learning-based	Node2Vec [15]	0.6580	0.6470	0.6650	0.4090	0.3760	0.4120	0.7190	0.7390	0.7650	0.4038	0.3741	0.3979
Labeling	SEAL [61]	0.2310	0.2020	0.3040	0.1640	0.3040	0.2800	0.3190	0.2800	0.4880	0.4901	0.4861	0.4959
	DRNL [61]	0.2230	0.2910	0.1770	0.2270	0.2050	0.2820	0.2070	0.2070	0.1800	0.2193	0.2006	0.4705
Structural	NCSM [27]	0.1400	0.1940	0.1710	0.1970	0.2280	0.2100	0.4970	0.5160	0.5080	0.2171	0.1132	0.2210
Positional	P-GNN [58]	0.4920	0.4720	0.3760	0.4140	0.4070	0.4040	0.6610	0.6740	0.6700	0.3639	0.3575	0.3592
	HPLC [22]	0.5420	0.5650	0.5530	0.4560	0.4830	0.4810	0.7110	0.7250	0.7090	0.4223	0.4112	0.4306
SCOUT (Ours)		0.7250	0.7160	0.7120	0.4960	0.4850	0.4920	0.7270	0.7490	0.7980	0.5071	0.5044	0.5183
GAIN		+10.18%	+10.66%	+7.07%	+8.77%	+0.41%	+2.29%	+1.11%	+1.35%	+4.31%	+3.47%	+3.76%	+4.52%
Node Classification (w/ orig. attr)		Cora (Acc)			Citeseer (Acc)			Pubmed (Acc)			ogbn-arxiv (Acc)		
only original attributes		0.7620	0.7880	0.7730	0.6860	0.6960	0.6870	0.7300	0.7660	0.7560	0.5336	0.5472	0.5443
Learning-based	Node2Vec [15]	0.8080	0.7980	0.6500	0.7100	0.7220	0.4400	0.7940	0.7730	0.7650	0.5752	0.5844	0.5752
Labeling	SEAL [61]	0.7760	0.7720	0.7880	0.7760	0.7840	0.7030	0.7990	0.7950	0.7830	0.5801	0.5882	0.6043
	DRNL [61]	0.7930	0.7800	0.7920	0.7840	0.7610	0.7770	0.7600	0.7780	0.7530	0.5498	0.5287	0.5709
Structural	NCSM [27]	0.8040	0.8050	0.7600	0.6830	0.6930	0.6440	0.7900	0.7620	0.7640	0.5403	0.5595	0.5646
Positional	P-GNN [58]	0.7100	0.6740	0.3620	0.6480	0.5930	0.4260	0.7690	0.7510	0.7100	0.5660	0.4764	0.3588
	HPLC [22]	0.6040	0.5670	0.5610	0.5420	0.5540	0.5350	0.7340	0.7540	0.7260	0.4899	0.4944	0.5306
SCOUT (Ours)		0.8140	0.8080	0.7930	0.7820	0.7840	0.7920	0.8070	0.8020	0.7930	0.6479	0.6577	0.6583
GAIN		+1.24%	+0.37%	+0.13%	+0.77%	+0.51%	+1.93%	+2.15%	+0.88%	+1.28%	+11.69%	+11.82%	+8.94%

Entries marked with '-' are not available because the dataset lacks original node features.

Different centrality–similarity pairs reveal distinct positional aspects, and the most informative aspect depends on the graph and task (see (D1) in Section 1). SCOUT addresses this dependency by (i) encoding multiple positional aspects into a single attribute vector. Because encoding every positional aspect can inject overcomplete information into the model, (ii) we learn a graph-level attention to select informative positional aspects. We compute the selection once per graph and share it across nodes; this fairness-driven design is empirically justified in RQ3.

We ablate the two components with two variants in Figure 5. Blue bars denote *Single-aspect*, where we use exactly one centrality–similarity pair and plot the top five pairs by AUC. For Cora, the top five are PG–RWR, PG–AdaSim, PG–Jaccard, PG–RA, BN–RWR. For

Citeseer, the top five are BN–RWR, BN–AdaSim, CN–RA, EV–RWR, PG–Jaccard. (Abbr. centrality: RD=random, DG=degree centrality, PG=PageRank, BN=betweenness centrality, CN=closeness, EV=eigenvector; similarity: RWR, AdaSim, RA, Jaccard.) Orange bars denote *Multi-aspect_{w/o Attn}*, where we encode all positional aspects but remove the selection module; predictions use a one-layer MLP on the concatenated attributes. Red bars denote SCOUT, which encodes all aspects and applies graph-level selection. Both (a) and (b) figures use the same backbone and the same K -selection rule for a controlled comparison for different datasets.

In *Multi-aspect_{w/o Attn}*, test AUC approaches the strongest single-aspect choice (e.g., PG–Jaccard) but is slightly lower, indicating that such overcomplete information dilutes task-relevant signal. With

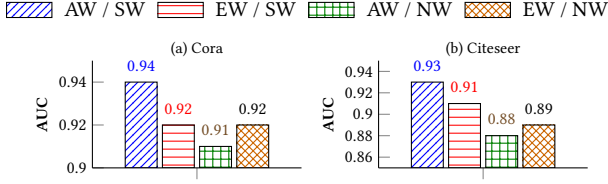


Figure 5: Design choice of the graph-level aspect-selection module. SCOUT attains the best results with the combination of graph-level shared weights and node-wise aggregation, validating our design.

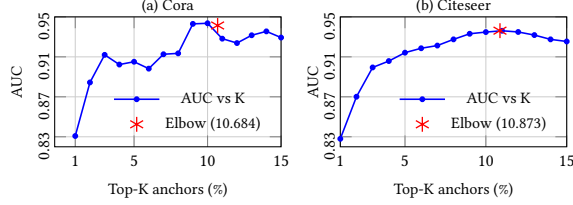


Figure 6: Effect of the anchor-count K and our selection rule on two datasets. SCOUT selects K per graph and stays close to the peak performance without manual search.

attention, SCOUT surpasses all single-aspect variants, showing that graph-level selection elevates informative aspects and suppresses distracting ones. Improvements beyond the best single pair imply genuine complementarity of aspect rather than simply selecting one aspect.

RQ3: Design Choice of the Graph-Level Aspect-Selection Module. The aspect-selection module in SCOUT operates *aspect-wise* (i.e., at the level of a centrality–similarity pair) and enforces a graph-level shared aspect mix by sharing the attention weights across nodes. We ablate two design axes: *aggregation granularity*—aspect-wise (AW) vs. element-wise (EW)—and *weighting scheme*—shared weights (SW) vs. node-specific weights (NW).

Along these axes, we compare three variants against SCOUT for two datasets in Figure 5. *AW/SW* corresponds to SCOUT. *AW/NW* removes weight sharing but keeps aspect-wise aggregation. *EW/SW* applies attention at attribute elements rather than aspects, while sharing weights. *EW/NW* combines both changes. As shown in Figure 5, *AW/SW* (SCOUT) attains the highest AUC. The gap to *AW/NW* in both datasets shows that graph-level sharing is beneficial when selection operates at the aspect level; enforcing a common aspect choice per graph stabilizes learning and avoids node-wise overfitting. *EW* variants underperform regardless of sharing, indicating that treating PI as coherent *aspects*—rather than independent elements—is the right abstraction for downstream tasks.

RQ4: Effectiveness of choosing the number of important nodes. We observe that the performance is sensitive to the anchor-count K , and the optimal K varies across graphs. We therefore determine K by applying the elbow rule to the rank-ordered centrality curve (Section 3.4). Figure 6 validates this choice by comparing AUC at our selected K against nearby values, including a sweep over top- K % anchors. On *Cora*, AUC peaks near 10% anchors, while our method selects 10.68%; the attained AUCs are virtually identical. Similar behavior holds on the other datasets, indicating that the rule

Table 2: Table 2: Runtime comparison. “Training & Inference” reports the cumulative wall-clock time summed over all epochs until training completes. Attribute generation dominates the end-to-end time for both methods. The runtime overhead of SCOUT is modest in practice.

Model	Calculate Elbow K (s)	Generate Attr. (s)	Training & Inference (s)	Total (s)
HPLC	-	3,691.83	75.06	3,766.89
SCOUT	1,977.00	2,292.03	167.04	4,436.07

chooses a near-optimal K without expensive search. This confirms that SCOUT adapts the anchor-count to each graph rather than relying on a fixed heuristic such as $K = O(\log |V|)$ [58].

RQ5: Computational cost. SCOUT computes PI for each centrality–similarity pair and encodes it as node attributes, which may raise concerns about runtime overhead. Table 2 breaks down wall-clock time on ogbn-arxiv into three stages: (i) Calculate Elbow, the additional cost of selecting a graph-adaptive anchor-count via elbow detection (SCOUT only); (ii) Generate Attr., the cost of generating PI attributes; and (iii) Training & Inference, the cumulative time of backbone training and inference summed over all epochs until training completes. Compared to HPLC [22], which uses a single pair, SCOUT uses $4 \times 4 = 16$ pairs, yet the total wall-clock overhead is only $\approx 17\%$. Overall, SCOUT scales to multi-aspect PI while maintaining competitive runtime.

The modest gap arises because modern GPUs accelerate sparse linear algebra and random-walk-based kernels, and recent implementations, e.g., cuGraph [46], compute diverse structural measures on GPUs efficiently. By leveraging these methods, SCOUT scales to multi-aspect PI while maintaining competitive wall-clock time and improving task performance.

5 Conclusion

We addressed the dependency of the PI-based method by selecting both the positional aspect and the anchor-count in a graph-aware manner. We leverage two architectures: (i) encoded multiple positional aspects and applied a graph-level attention that shares the weight across nodes, (ii) detect the elbow on the ranked centrality distribution, which follows a heavy-tailed behavior, to determine the anchor count K . Across citation graphs and Open Graph Benchmark datasets, SCOUT improved link prediction and node classification, with consistent gains across all datasets and tasks, with or without original attributes. These results support SCOUT as a model-agnostic augmentation that complements strong backbones. In summary, by coupling graph-level aspect selection with data-driven anchor selection, SCOUT turns PI augmentation into an adaptable component for graph neural networks.

Acknowledgments

This work was supported by Institute of Information communications Technology Planning Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2022-00155586 and No.RS-2020-II201373) and by the National Research Foundation of Korea (NRF) funded by the Korea Government (MSIT) under Grant RS-2021-NR059723.

References

- [1] Sameer Anand, Rahul, Abhishek Mallik, and Sanjay Kumar. 2022. Integrating node centralities, similarity measures, and machine learning classifiers for link prediction. *Multimedia tools and applications* 81, 27 (2022), 38593–38621.
- [2] Alex Bavelas. 1950. Communication Patterns in Task-Oriented Groups. *The Journal of the Acoustical Society of America* 22, 6 (1950), 725–730. doi:10.1121/1.1906679
- [3] Phillip Bonacich. 1987. Power and Centrality: A Family of Measures. *Amer. J. Sociology* 92, 5 (1987), 1170–1182. doi:10.1086/228631
- [4] Karsten M Borgwardt and Hans-Peter Kriegel. 2005. Shortest-path kernels on graphs. In *Fifth IEEE international conference on data mining (ICDM'05)*. IEEE, 8–pp.
- [5] Jean Bourgain. 1985. On Lipschitz embedding of finite metric spaces in Hilbert space. *Israel Journal of Mathematics* 52, 1 (1985), 46–52.
- [6] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual web search engine. *Computer networks and ISDN systems* 30, 1–7 (1998), 107–117.
- [7] Eli Chien, Wei-Ning Chen, Chao Pan, Pan Li, Ayfer Özgür, and Olgica Milenkovic. 2023. Differentially Private Decoupled Graph Convolutions for Multigranular Topology Protection. arXiv:2307.06422 [cs.LG] <https://arxiv.org/abs/2307.06422>
- [8] Aaron Clauset, Cosma Rohilla Shalizi, and Mark EJ Newman. 2009. Power-law distributions in empirical data. *SIAM review* 51, 4 (2009), 661–703.
- [9] Chi Thang Duong, Thanh Dat Hoang, Ha The Hien Dang, Quoc Viet Hung Nguyen, and Karl Aberer. 2019. On node features for graph neural networks. *arXiv preprint arXiv:1911.08795* (2019).
- [10] Vijay Prakash Dwivedi and Xavier Bresson. 2021. A Generalization of Transformer Networks to Graphs. arXiv:2012.09699 [cs.LG] <https://arxiv.org/abs/2012.09699>
- [11] Linton C Freeman. 1977. A set of measures of centrality based on betweenness. *Sociometry* (1977), 35–41.
- [12] C Lee Giles, Kurt D Bollacker, and Steve Lawrence. 1998. CiteSeer: An automatic citation indexing system. In *Proceedings of the third ACM conference on Digital libraries*. 89–98.
- [13] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. Pmlr, 1263–1272.
- [14] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. Pmlr, 1263–1272.
- [15] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.
- [16] Masoud Reyhani Hamedani and Sang-Wook Kim. 2021. AdaSim: A Recursive Similarity Measure in Graphs. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*. ---. doi:10.1145/3459637.3482316
- [17] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [18] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [19] Wen Huang, Yu Rong, Tingyang Xu, Fuchun Sun, and Junzhou Huang. 2022. Node Position Encoding for Graph Neural Networks. In *International Conference on Learning Representations (ICLR)*.
- [20] Paul Jaccard. 1912. The distribution of the flora in the alpine zone. 1. *New phytologist* 11, 2 (1912), 37–50.
- [21] Bo Jiang, Ziyao Zhang, Doudou Lin, Jin Tang, and Bin Luo. 2019. Semi-supervised learning with graph learning-convolutional networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11313–11320.
- [22] Minsang Kim and Seung Baek. 2024. Hierarchical Position Embedding of Graphs with Landmarks and Clustering for Link Prediction. In *Proceedings of the ACM Web Conference 2024*. 389–400.
- [23] Taeri Kim, Sohee Ban, Hyunjoon Kim, and Sang-Wook Kim. 2025. STARLINE: Contrastive Learning with Modality-Aware Graph Refinement for Effective Multimedia Recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 1184–1195. doi:10.1145/3711896.3737136
- [24] Yungi Kim, Taeri Kim, Won-Yong Shin, and Sang-Wook Kim. 2024. MONET: Modality-Embracing Graph Convolutional Network and Target-Aware Attention for Multimedia Recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining* (Merida, Mexico) (WSDM '24). Association for Computing Machinery, New York, NY, USA, 332–340. doi:10.1145/3616855.3635817
- [25] TN Kipf. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [26] Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. 2020. Distance Encoding: Design Provably More Powerful Neural Networks for Graph Representation Learning. arXiv:2009.00142 [cs.LG] <https://arxiv.org/abs/2009.00142>
- [27] Haohui Lu and Shahadat Uddin. 2024. A parameterised model for link prediction using node centrality and similarity measure based on graph embedding. *Neurocomputing* 593 (2024), 127820.
- [28] Li Ma, Haoyu Han, Juanhui Li, Harry Shomer, Hui Liu, Xiaofeng Gao, and Jiliang Tang. 2024. Mixture of link predictors on graphs. *Advances in Neural Information Processing Systems* 37 (2024), 16043–16070.
- [29] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* 3, 2 (2000), 127–163.
- [30] Xupeng Miao, Nezihe Merve Gürel, Wentao Zhang, Zhichao Han, Bo Li, Wei Min, Susie Xi Rao, Hansheng Ren, Yanan Shan, Yingxia Shao, et al. 2021. Degnn: Improving graph neural networks with graph decomposition. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 1223–1233.
- [31] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. 2018. Weisfeiler and Leman Go Neural: Higher-order Graph Neural Networks. *CoRR* abs/1810.02244 (2018). arXiv:1810.02244 <http://arxiv.org/abs/1810.02244>
- [32] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. 2019. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 4602–4609.
- [33] Galileo Namata, Ben London, Lise Getoor, Bert Huang, and U Edu. 2012. Query-driven active surveying for collective classification. In *10th international workshop on mining and learning with graphs*, Vol. 8. 1.
- [34] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 701–710.
- [35] Ladislav Rampáček, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2023. Recipe for a General, Powerful, Scalable Graph Transformer. arXiv:2205.12454 [cs.LG] <https://arxiv.org/abs/2205.12454>
- [36] Leonardo F.R. Ribeiro, Pedro H.P. Saverese, and Daniel R. Figueiredo. 2017. struc2vec: Learning Node Representations from Structural Identity. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '17)*. ACM, 385–394. doi:10.1145/3097983.3098061
- [37] Anwar Said, Mudassir Shabbir, Tyler Derr, Waseem Abbas, and Xenofon Koutsoukos. 2023. Enhanced Graph Neural Networks with Ego-Centric Spectral Subgraph Embeddings Augmentation. arXiv:2310.12169 [cs.LG] <https://arxiv.org/abs/2310.12169>
- [38] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. 2011. Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. In *2011 31st International Conference on Distributed Computing Systems Workshops*. 166–171. doi:10.1109/ICDCSW.2011.20
- [39] Dong-Hyuk Seo, Jae-Hwan Lim, Won-Yong Shin, and Sang-Wook Kim. 2024. Leveraging Trustworthy Node Attributes for Effective Network Alignment (CIKM '24). Association for Computing Machinery, New York, NY, USA, 2004–2013. doi:10.1145/3627673.3679658
- [40] Dong-Hyuk Seo, Hyomin Shin, and Sang-Wook Kim. 2025. Exploring Search Volumes of Terms in Web Portals for Accurate Event-Aware Traffic Prediction. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3701716.3715582
- [41] Marvin E Shaw. 1954. Group structure and the behavior of individuals in small groups. *The Journal of psychology* 38, 1 (1954), 139–149.
- [42] C. Shi et al. 2021. A quantitative discriminant method of elbow point for the determination of optimal cluster number. *EURASIP Journal on Wireless Communications and Networking* (2021). doi:10.1186/s13638-021-01910-w
- [43] Lei Shi, Bin Hu, Deng Zhao, Jianshan He, Zhiqiang Zhang, and Jun Zhou. 2024. Structural Information Enhanced Graph Representation for Link Prediction. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 13 (Mar. 2024), 14964–14972. doi:10.1609/aaai.v38i13.29417
- [44] Yundong Sun, Dongjie Zhu, Haiwen Du, and Zhaoshuo Tian. 2022. MHNF: Multi-hop heterogeneous neighborhood information fusion graph representation learning. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2022), 7192–7205.
- [45] Hibiki Taguchi, Xin Liu, and Tsuyoshi Murata. 2021. Graph convolutional networks for graphs containing missing features. *Future Generation Computer Systems* 117 (April 2021), 155–168. doi:10.1016/j.future.2020.11.016
- [46] RAPIDS Development Team. 2023. *RAPIDS: Libraries for End to End GPU Data Science*. <https://rapids.ai>
- [47] Komal K. Teru and William L. Hamilton. 2019. Inductive Relation Prediction on Knowledge Graphs. *CoRR* abs/1911.06962 (2019). arXiv:1911.06962 <http://arxiv.org/abs/1911.06962>
- [48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [49] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks.

- arXiv:1710.10903 [stat.ML] <https://arxiv.org/abs/1710.10903>
- [50] Qian Wang, Jiaxing Zhang, Sen Song, and Zheng Zhang. 2014. Attentional Neural Network: Feature Selection Using Cognitive Feedback. arXiv:1411.5140 [cs.CV] <https://arxiv.org/abs/1411.5140>
- [51] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
- [52] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. arXiv:1906.00121 [cs.LG] <https://arxiv.org/abs/1906.00121>
- [53] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks? arXiv:1810.00826 [cs.LG] <https://arxiv.org/abs/1810.00826>
- [54] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do Transformers Really Perform Bad for Graph Representation? arXiv:2106.05234 [cs.LG] <https://arxiv.org/abs/2106.05234>
- [55] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 974–983.
- [56] Jiaxuan You, Jonathan M Gomes-Selman, Rex Ying, and Jure Leskovec. 2021. Identity-aware graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35, 10737–10745.
- [57] Jiaxuan You, Bowen Liu, Rex Ying, Vijay Pande, and Jure Leskovec. 2019. Graph Convolutional Policy Network for Goal-Directed Molecular Graph Generation. arXiv:1806.02473 [cs.LG] <https://arxiv.org/abs/1806.02473>
- [58] Jiaxuan You, Rex Ying, and Jure Leskovec. 2019. Position-aware graph neural networks. In *International conference on machine learning*. PMLR, 7134–7143.
- [59] Jiaxuan You, Rex Ying, and Jure Leskovec. 2019. Position-aware Graph Neural Networks. arXiv:1906.04817 [cs.LG] <https://arxiv.org/abs/1906.04817>
- [60] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2017. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *arXiv preprint arXiv:1709.04875* (2017).
- [61] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in neural information processing systems* 31 (2018).
- [62] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski. 2019. Graph convolutional networks: a comprehensive review. *Computational Social Networks* 6, 1 (2019), 1–23.
- [63] Tao Zhou, Linyuan Lü, and Yi-Cheng Zhang. 2009. Predicting Missing Links via Local Information. *The European Physical Journal B* 71, 4 (2009), 623–630. doi:10.1140/epjb/e2009-00335-8 Introduces the Resource Allocation (RA) index.
- [64] Wenhao Zhu, Guojie Song, Liang Wang, and Shaoguo Liu. 2024. Anchorgt: Efficient and flexible attention architecture for scalable graph transformers. *arXiv preprint arXiv:2405.03481* (2024).
- [65] Marinka Zitnik, Monica Agrawal, and Jure Leskovec. 2018. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics* 34, 13 (06 2018), i457–i466. arXiv:https://academic.oup.com/bioinformatics/article-pdf/34/13/i457/50316205/bioinformatics_34_13_457.pdf doi:10.1093/bioinformatics/bty294
- [66] Marinka Zitnik, Monica Agrawal, and Jure Leskovec. 2018. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics* 34, 13 (2018), i457–i466.
- [67] Marinka Zitnik, Monica Agrawal, and Jure Leskovec. 2018. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics* 34, 13 (June 2018), i457–i466. doi:10.1093/bioinformatics/bty294

Appendix A: Datasets Description

Table 3: Dataset statistics used in SCOUT. Citation graphs follow the Planetoid features; ogbl-ddi has no node features; ogbn-arxiv provides 128-dim attributes. Official splits and metrics follow our experimental setup.

Dataset	$ V $ (nodes)	$ E $ (edges)	Orig. attr.	Feat. dim d
Cora	2,708	5,429	yes	1,433
Citeseer	3,327	4,732	yes	3,703
Pubmed	19,717	44,338	yes	500
ogbl-ddi	4,267	1,334,889	no	–
ogbn-arxiv	169,343	1,166,243	yes	128

Appendix B: Notations

Table 4: Table of notations.

Symbol	Description
$\mathcal{G} = (V, E)$	Graph with node set V and edge set E
$ V = N$	Number of nodes
$ E $	Number of edges
K	Number of anchors (Anchor count)
ℓ	Number of centrality measures
m	Number of similarity measures
$c^{(i)} : V \rightarrow \mathbb{R}_{\geq 0}$	i -th centrality function
$s^{(j)} : V \times V \rightarrow \mathbb{R}_{\geq 0}$	j -th node-pair similarity
$R^{(i)} = \{r_1^{(i)}, \dots, r_K^{(i)}\} \subseteq V$	Anchor set for the i -th centrality
$\tilde{X} \in \mathbb{R}^{tmK}$	Positional attribute of node n
$X^{orig} \in \mathbb{R}^{ V \times d}$	Original node attribute matrix (attribute dimension d)
$\ $	Concatenation operator

Appendix C: Centrality and Similarity used in SCOUT

Centrality Measures.

Degree centrality. Degree centrality uses the node degree $d(v)$ as the score [41]. High-score nodes are hubs that connect to many neighbors, often inside dense cores, while leaves rank low. We normalize by $\tilde{c}(v) = d(v) / \max_{u \in V} d(u)$ to stabilize across graphs. Computation costs $O(|E|)$ for unweighted graphs. Ties are frequent in sparse graphs, so we break ties by secondary metrics only when needed.

PageRank centrality. PageRank solves $\pi^\top = \alpha \pi^\top P + (1 - \alpha) u^\top$ with damping $\alpha \in (0, 1)$ and teleport u . High-score nodes are those cited by high-score neighbors, so authority flows accumulate on well-endorsed nodes, not merely on high-degree ones. We use L_1 normalization to interpret scores as probabilities. Power iteration costs $O(T|E|)$ where T is the iteration count to tolerance. Dangling nodes use standard row-replacement which prevents sink traps in anchor selection.

Betweenness centrality. Betweenness counts shortest-path flow through a node, $c(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$. High-score nodes are bridges that many shortest paths traverse, typically at community boundaries or articulation points. We rescale by $\tilde{c}(v) = c(v) / \max_u c(u)$ to compare across datasets. Exact Brandes runs in $O(|V||E|)$ on unweighted graphs, so we use sampling when $|V|$ is large. Sampling reduces variance as sample size grows, so anchor-count decisions remain stable.

Closeness centrality. Closeness uses the inverse of average shortest-path distance, $c(v) = \frac{|V|-1}{\sum_{u \neq v} d(v,u)}$. High-score nodes sit in the geometric middle of the graph, reaching others with few hops, while peripheral nodes rank low. We use harmonic closeness in disconnected graphs to avoid infinities. SSSP-based computation costs $O(|V||E|)$ unweighted, so we adopt BFS sampling when needed. Closeness is sensitive to graph diameter, so normalization helps cross-graph comparability.

Similarity measures.

Random-walk with restart (RWR). RWR solves $x = \beta e_v + (1 - \beta)P^T x$ for source v and restart $\beta \in (0, 1)$. Perceived closeness is high when many short and moderately long walks from v visit u , so multi-hop but well-connected routes matter. We row-normalize P and L_1 -normalize x to keep scores comparable across anchors. Power iteration costs $O(T|E|)$ per source, so we reuse factorizations where possible. RWR is robust to mild noise, so positional information remains smooth yet discriminative.

AdaSim. AdaSim adapts walk length and neighborhood weighting to degree, so similarity de-biases hubs. Perceived closeness is high when nodes co-occur along walks that avoid popular hubs and emphasize rare, informative connectors. We clip and z-score at the graph level to align scales with other similarities. Precomputation uses sparse-matrix multiplications with cost $O(T|E|)$. Adaptive weighting improves ranking near anchors in heterophilous graphs.

Resource Allocation (RA). RA similarity uses common neighbors with inverse degree weights, $s(u, v) = \sum_{w \in \Gamma(u) \cap \Gamma(v)} \frac{1}{d(w)}$. Perceived closeness is high when u and v share low-degree intermediates, so rare shared neighbors contribute more than hubs. We min-max scale RA per anchor to mitigate graph-density effects. A naive implementation costs $O(\sum_u d(u)^2)$, and sparse intersection reduces runtime. RA complements RWR by emphasizing local exclusivity over diffuse proximity.

Jaccard coefficient. Jaccard uses set overlap, $s(u, v) = \frac{|\Gamma(u) \cap \Gamma(v)|}{|\Gamma(u) \cup \Gamma(v)|}$. Perceived closeness is high when neighborhoods overlap strongly relative to their union, so balanced overlap outranks sheer size. We compute on pruned k -NN adjacency when graphs are dense to save memory. Sorted-list intersection yields near-linear cost in neighborhood sizes. Jaccard is scale bounded in $[0, 1]$, so it serves as a stable baseline for aspect selection.

Appendix D: Accuracy Measures used in SCOUT

Area Under the ROC Curve (AUC). AUC measures the probability that a random positive edge receives a higher score than a random negative edge, so larger AUC indicates better pairwise ranking ability. We compute scores for held-out positive edges and sampled negatives under the same split and optimize binary cross-entropy during training, so evaluation aligns with the training objective. We report AUC for citation-graph link prediction to match prior work and enable fair comparison.

Average Precision (AP). AP is the area under the precision-recall curve computed by ranking candidate edges by score, so it emphasizes performance on the positive class when negatives dominate. We aggregate precision at every recall level induced by the ranked list, so AP captures the full retrieval curve rather than a single threshold. We include AP on citation graphs to complement AUC under class imbalance.

Hits@20. Hits@20 is the fraction of test queries whose true edge appears in the top 20 predicted candidates, so it evaluates top- K ranking quality. We adopt Hits@20 on ogbl-ddi because the OGB benchmark defines it as the official metric, and most studies follow this protocol. We generate the ranked list per source node under the official evaluator, so results are directly comparable to prior work.

Classification Accuracy. Accuracy is the proportion of correctly predicted labels among test nodes, so it summarizes overall classification correctness. We use cross-entropy for single-label and binary cross-entropy for multi-label cases during training, then report accuracy on the official split for node classification. We follow the OGB protocol for large graphs and a 70/10/20 train/val/test split for citation graphs to ensure comparability.

Appendix E: Empirical Validation of Similarity Dependence (D1)

To validate the claim D1 stated in Section 1, we empirically analyze the dependence of performance on the choice of similarity measures. We fix the anchor selection criterion to *PageRank* and compare the link prediction performance (AUC) using four different similarity measures: RWR, AdaSim, RA, and Jaccard.

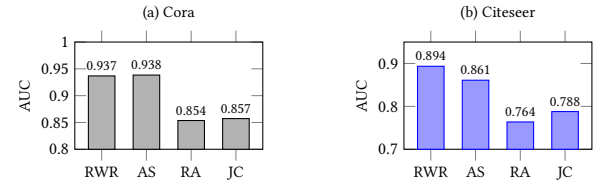


Figure 7: Link prediction performance (AUC) across different similarity measures. The optimal similarity measure shifts from AdaSim (AS) in Cora to RWR in Citeseer, confirming that structural measure selection is highly graph-dependent.

As shown in Figure 7, the performance varies significantly depending on the similarity measure used. Specifically, *AdaSim* achieves the best performance on Cora, whereas *RWR* outperforms others on Citeseer. This significant performance gap (up to 12.9%p in Citeseer) and the shift in the optimal measure across datasets justify the necessity of SCOUT’s adaptive graph-level aspect selection mechanism.

Appendix F: Design Choice of Anchor-count K

Table 5: Link prediction on citation graphs: AUC and AP under two anchor-count selection settings for SCOUT. Larger is better.

Setting	Cora		Citeseer		PubMed	
	AUC	AP	AUC	AP	AUC	AP
Single k	0.9434	0.9559	0.9349	0.9381	0.9588	0.9701
Different k	0.9422	0.9320	0.9025	0.9162	0.9510	0.9455

We fix the positional aspects and the backbone, then vary only the anchor-count strategy between *Single K* and *Different K* . Single K chooses K once by the elbow on the ranked centrality curve and shares it across aspects. Different K chooses a separate K for each centrality, then pads or projects aspect vectors to align dimensions for *graph-level aspect selection*.

We report AUC and AP for link prediction on Cora, Citeseer, and PubMed. As shown in the Table, *Single K*, which is our design choice, matches or exceeds *Different K* across datasets. It is because *Different K* yields aspect vectors of unequal length, aspect-wise aggregation requires additional transforms to align dimensions, which introduces extra parameters and destabilizes optimization.

Appendix G: Comparison using State-of-the-Art Methods as Backbones

Table 6: AUC comparison on link prediction without original node attributes. We compare SCOUT and HPLC when applied to GCN and various Graph Transformer backbones.

Model	GCN	Graph Transformer	Graphormer	GPS
1) Cora				
SCOUT	0.9434	0.9492	0.9213	0.9483
HPLC	0.9135	0.9250	0.9117	0.9193
2) Citeseer				
SCOUT	0.9349	0.9364	0.9200	0.9335
HPLC	0.9293	0.9323	0.9160	0.9217
3) Pubmed				
SCOUT	0.9588	0.9291	0.9551	0.9683
HPLC	0.9327	0.9237	0.9053	0.9297

To demonstrate the universality of SCOUT, we evaluate it on state-of-the-art Graph Transformer backbones: Graph Transformer [10], Graphormer [54], and GPS [35]. As shown in Table 6, SCOUT consistently outperforms the strongest baseline, HPLC [22], across all datasets. Notably, SCOUT further enhances powerful models like GPS (e.g., 0.9683 AUC on Pubmed), verifying that our structure-aware aspect selection offers complementary gains as a robust, model-agnostic augmentation.